

第 8 章 类和对象

第二章的通讯录是一个列表套字典，加人删人都得写一堆代码，而且谁都能随手改坏里面的数据。这一章用类把数据和操作打包在一起，重写这个通讯录。

8.1 先用类重写通讯录

本节任务：把通讯录做成一个类，数据和增删改查都装在里面，外面用起来只有几行。

```
1 class Contact:
2     """一个联系人"""
3     def __init__(self, name, phone):
4         self.name = name
5         self.phone = phone
6
7     def __repr__(self):
8         return f"{self.name} {self.phone}"
9
10
11 class AddressBook:
12     """一本通讯录"""
13     def __init__(self):
14         self.people = []
15
16     def add(self, name, phone):
17         self.people.append(Contact(name, phone))
18
19     def find(self, name):
20         for p in self.people:
21             if p.name == name:
22                 return p
23         return None
24
25     def remove(self, name):
26         p = self.find(name)
27         if p:
28             self.people.remove(p)
29         return True
```

```
30     return False
31
32     def __len__(self):
33         return len(self.people)
34
35
36 book = AddressBook()
37 book.add("小明", "13800001111")
38 book.add("小红", "13900002222")
39 book.add("小刚", "13700003333")
40
41 print("共", len(book), "人")
42 print("查小红: ", book.find("小红"))
43 print("删小刚: ", book.remove("小刚"))
44 print("现在: ", book.people)
```

运行结果:

```
共 3 人
查小红:  小红 13900002222
删小刚:  True
现在:  [小明 13800001111, 小红 13900002222]
```

和第二章比，好处在于用起来变简单了。加人是 `book.add`，查人是 `book.find`，连 ★核心数人数都能直接用 `len`。怎么存、怎么找这些细节被藏在类里面，外面不用关心。下面把类的各个部分讲清楚。

8.2 类和实例

本节任务：分清类和实例，会定义最简单的类。

类是一个模板，描述某一类东西长什么样、能做什么。按模板造出来的具体东西叫**实例**。上面 `Contact` 是类，小明和小红是两个实例。

定义类用 `class`，类名习惯用大写字母开头：

```
1 class Dog:
2     def bark(self):
3         print("汪")
4
5 d = Dog() # 造一个实例
6 d.bark() # 汪
```

类里定义的函数叫**方法**，第一个参数固定是 **self**，代表调用它的那个实例本身。调 ★核心用时不用传，Python 会自动把实例塞进去。写方法忘了写 **self** 是最常见的错误之一。

8.3 init 和属性

本节任务：让每个实例带上自己的数据。

光有方法还不够，每个实例得有自己的数据。这靠 **__init__** 方法，它在造实例时自动被调用，用来把初始数据装上去：

```
1 class Dog:
2     def __init__(self, name, age):
3         self.name = name # 挂到实例上
4         self.age = age
5
6     def intro(self):
7         print(f"我叫{self.name}, {self.age}岁")
8
9 a = Dog("旺财", 3)
10 b = Dog("小黑", 5)
11 a.intro() # 我叫旺财, 3岁
12 b.intro() # 我叫小黑, 5岁
13 print(a.name) # 旺财 也可以直接取
```

挂在 **self** 上的数据叫**属性**。两个实例各有各的 **name** 和 **age**，互不影响。造实例时写在括号里的参数，就是传给 **__init__** 的。

前后各两条下划线的方法有特殊含义，由 Python 在特定时机自动调用，不用自己写 **__init__** 加括号去调。

8.4 几个有用的特殊方法

本节任务：让自己的类用起来像内置类型一样自然。

除了 **__init__**，还有几个常用的特殊方法。开头的通讯录用了两个。

__repr__ 决定实例被打印出来的样子。不写的话打印出来是一串没意义的地址：

```
1 class Contact:
2     def __init__(self, name, phone):
3         self.name, self.phone = name, phone
4
```

```
5 c = Contact("小明", "138")
6 print(c) # <__main__.Contact object at 0x000001...> 看不出是谁
```

加上 `__repr__` 就清楚了:

```
1 class Contact:
2     def __init__(self, name, phone):
3         self.name, self.phone = name, phone
4     def __repr__(self):
5         return f"{self.name} {self.phone}"
6
7 print(Contact("小明", "138")) # 小明 138
```

`__len__` 让实例支持 `len`。开头的通讯录写了它，所以能直接 `len(book)`。类似的还有 `__eq__` 决定两个实例怎么算相等，`__getitem__` 让实例支持方括号取值。用到时再查即可。

8.5 继承，站在已有的类上改

本节任务：从已有的类派生出新类，只写不一样的部分。

如果新类和已有的类大部分一样，只有少数不同，用**继承**：

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4     def speak(self):
5         print("...")
6     def intro(self):
7         print(f"我是{self.name}")
8
9 class Dog(Animal): # 括号里写父类
10     def speak(self): # 只改这个方法
11         print("汪")
12
13 class Cat(Animal):
14     def speak(self):
15         print("喵")
16
17 for a in [Dog("旺财"), Cat("咪咪")]:
18     a.intro() # 从 Animal 继承来的
```

```
19 a.speak() # 各自的
```

输出：

```
我是旺财
汪
我是咪咪
喵
```

Dog 和 Cat 没写 `__init__` 和 `intro`，直接用父类的。`speak` 各自重写了一份，这叫**重写**。

如果子类也要有自己的初始化，记得先调用父类的：

★核心

```
1 class Dog(Animal):
2     def __init__(self, name, breed):
3         super().__init__(name) # 先让父类把 name 装好
4         self.breed = breed
```

`super()` 指代父类。漏掉这一句，父类里做的初始化就不会执行，`self.name` 也就不存在了。

8.6 什么时候该用类

本节任务：知道什么情况值得写类，什么情况没必要。

不是所有代码都该写成类。判断标准是**数据和操作是不是绑在一起的**。

该用类的信号有两个。一是你发现一堆函数总是在操作同一份数据，比如 `add_contact`、`find_contact`、`remove_contact` 都在摆弄同一个列表，那这个列表和这些函数就该打包成一个类。二是你需要同时存在好几份互不干扰的同类东西，比如两本通讯录，各自有各自的数据。

不该用类的情况也很常见。只是算个数、转换一下格式，写个函数就够了，包成类反而绕。第四章说过一个函数只做一件事，同理，一个类也应该只描述一样东西。

第二章的做法和这一章的做法都对，数据少时列表套字典更直接。等到操作变多、需要保证数据不被随手改坏时，类的好处才显出来。

8.7 小结

这一章把数据和操作打包在了一起。类是模板，实例是按模板造出来的具体东西。方法的第一个参数固定是 `self`，代表实例自己。`__init__` 在造实例时自动调用，用来装上初始数据，挂在 `self` 上的数据叫属性。`__repr__` 决定打印出来的样子，`__len__`

让实例支持 `len`。继承能从已有的类派生新类，只写不一样的部分，子类有自己的初始化时要用 `super` 先调父类。一堆函数总在操作同一份数据，就该考虑写成类。下一章开始接触数据处理，从数组讲起。

本章知识点

- **类**——描述某一类东西长什么样、能做什么的模板，类名首字母大写
- **实例**——按类造出来的具体东西，各有各的数据
- **方法**——类里定义的函数，第一个参数固定是 `self`
- **self**——代表调用方法的那个实例本身，调用时不用传
- **`__init__`**——造实例时自动调用，用来装上初始数据
- **属性**——挂在 `self` 上的数据，每个实例各有一份
- **`__repr__`**——决定实例打印出来的样子，不写就是一串地址
- **`__len__`**——让实例支持 `len` 函数
- **继承**——从已有的类派生新类，只写不一样的部分
- **重写**——子类重新定义父类已有的方法
- **`super`**——指代父类，子类有自己的初始化时先调它